This article was downloaded by: [Colorado College]
On: 14 October 2014, At: 16:34
Publisher: Taylor & Francis
Informa Ltd Registered in England and Wales Registered Number: 1072954 Registered office: Mortimer House, 37-41 Mortimer Street, London W1T 3JH, UK



International Journal of Digital Earth
Publication details, including instructions for authors and subscription information:
http://www.tandfonline.com/loi/tjde20

Parallel algorithm for viewshed analysis on a modern GPU

Fang Chao [a], Yang Chongjun [a], Chen Zhuo [a], Yao Xiaojing [a] & Guo Hantao [b]

[a] State Key Laboratory of Remote Sensing Science, Jointly Sponsored by The Institute of Remote Sensing Applications of Chinese Academy of Sciences and Beijing Normal University, Beijing, China

[b] Department of 3D product, Beijing GeoBeans Information Technology Co., Ltd, Beijing, China

Published online: 17 Feb 2011.

To cite this article: Fang Chao, Yang Chongjun, Chen Zhuo, Yao Xiaojing & Guo Hantao (2011) Parallel algorithm for viewshed analysis on a modern GPU, International Journal of Digital Earth, 4:6, 471-486, DOI: 10.1080/17538947.2011.555565

To link to this article: http://dx.doi.org/10.1080/17538947.2011.555565

PLEASE SCROLL DOWN FOR ARTICLE

Taylor & Francis makes every effort to ensure the accuracy of all the information (the "Content") contained in the publications on our platform. However, Taylor & Francis, our agents, and our licensors make no representations or warranties whatsoever as to the accuracy, completeness, or suitability for any purpose of the Content. Any opinions and views expressed in this publication are the opinions and views of the authors, and are not the views of or endorsed by Taylor & Francis. The accuracy of the Content should not be relied upon and should be independently verified with primary sources of information. Taylor and Francis shall not be liable for any losses, actions, claims, proceedings, demands, costs, expenses, damages, and other liabilities whatsoever or howsoever caused arising directly or indirectly in connection with, in relation to or arising out of the use of the Content.

This article may be used for research, teaching, and private study purposes. Any substantial or systematic reproduction, redistribution, reselling, loan, sub-licensing, systematic supply, or distribution in any form to anyone is expressly forbidden. Terms &

Conditions of access and use can be found at http://www.tandfonline.com/page/terms-and-conditions

Downloaded by [Colorado College] at 16:34 14 October 2014

International Journal of Digital Earth,
Vol. 4, No. 6, November 2011, 471–486

Taylor & Francis
Taylor & Francis Group

Parallel algorithm for viewshed analysis on a modern GPU

Fang Chao[a]*, Yang Chongjun[a], Chen Zhuo[a], Yao Xiaojing[a] and Guo Hantao[b]

[a]*State Key Laboratory of Remote Sensing Science, Jointly Sponsored by The Institute of Remote Sensing Applications of Chinese Academy of Sciences and Beijing Normal University, Beijing, China;* [b]*Department of 3D product, Beijing GeoBeans Information Technology Co., Ltd, Beijing, China*

(*Received 19 October 2010; final version received 12 January 2011*)

Spatial analysis, including viewshed analysis, is an important aspect of the Digital Earth system. Viewshed analysis is usually performed on a large scale, so efficiency is important in any Digital Earth application making these calculations. In this paper, a real-time algorithm for viewshed analysis in 3D scenes is presented by using the parallel computing capabilities of a graphics processing unit (GPU). In contrast to traditional algorithms based on line-of-sight, this algorithm runs completely within the programmable 3D visualization pipeline to render 3D terrains with viewshed analysis. The most important difference is its integration of the viewshed calculation with the rendering module. Invisible areas are rendered as shadows in the 3D scene. The algorithm process is paralleled by rasterizer units in the graphics card and by vertex and pixel shaders executed on the GPU. We have implemented this method in our 3D Digital Earth system with the DirectX 9.0c API and tested on some consumer-level PC platforms with interactive frame-rates and high image quality. Our algorithm has been widely used in related systems based on Digital Earth.

Keywords: viewshed analysis; vertex and pixel shader; GPU; shadow map; 3D representation; visualization; Digital Earth; DEM

Downloaded by [Colorado College] at 16:34 14 October 2014

1. Introduction

Digital Earth is a complex system and platform for digitally representing and analyzing geographically relevant information from around the world in three dimensions. In addition to the primary function of displaying the world's topography and remote sensing imagery, Digital Earth can also involve various spatial analysis applications such as viewshed, buffer, and shortest path analyses. Many government departments use professional systems based on Digital Earth, such as land management or detection systems, military information systems, and 3D city planning information systems. For these applications, spatial analysis is especially important to assist in decision-making. Many modules in these applications are derived from viewshed analysis. Viewshed analysis is a process for deriving the portion of the land's surface visible from one or more viewpoints (Chang 2005). It is commonly used for site selection, such as for forest lookout stations in land detection systems, cellular telephone base stations in 3D city planning information systems, and to cover defense in military information systems. Viewshed analysis has become

*Corresponding author. Email: magicianfc@gmail.com

ISSN 1753-8947 print/ISSN 1753-8955 online
© 2011 Taylor & Francis
http://dx.doi.org/10.1080/17538947.2011.555565
http://www.tandfonline.com

an essential function of geographic information systems (GIS) and an indispensable part of Digital Earth.

Traditional viewshed analysis algorithms often require gridded digital elevation models (DEMs) and usually display the result in 2D form. In these algorithms, the viewshed calculation is independent from the rendering module. This study focuses on the following two points: how to conduct a viewshed analysis in 3D form in a Digital Earth system and how to integrate the viewshed calculation into a 3D rendering module. In contrast to the traditional algorithms executed on a computer's central processing unit (CPU), the algorithm demonstrated in this paper is a real-time parallel algorithm completely executed on a modern graphics processing unit (GPU) with a programmable 3D visualization pipeline. A GPU is a microprocessor dedicated to rendering graphics. High-end GPUs are usually built onto graphics cards (also called video cards), which can have dedicated memory and other supporting components.

Section 2 gives a brief overview and discussion of previous work on viewshed analysis and shadow mapping, a classic algorithm to generate shadows in 3D computer graphics. Then in Section 3, we introduce a new concept to simulate viewshed analysis using a light source to illuminate and cast shadows on the terrain. The details of this algorithm, which contains three passes, are presented in Section 4. The advantages of this algorithm are described in Section 5, and the performance and quality issues are discussed in Section 6. Finally, we give a short conclusion and consideration of future prospects in Section 7.

Downloaded by [Colorado College] at 16:34 14 October 2014

2. A brief overview of previous work and discussion

2.1 Introduction to intervisibility

Intervisibility involves finding the visibility between a viewpoint and a target point and is the basis of viewshed analysis. The essence of intervisibility is the line-of-sight operation. For a viewpoint O and a target point A, a line segment between O and A is created to make up the line of sight. Moving along the line segment OA and testing all of the points on it, if there is a point P with an elevation lower than the elevation of the corresponding point on the terrain, target point A is invisible to the viewpoint O. This requires much computation and its time complexity is expressed as $O(n^2)$.

This algorithm comprises six steps:

(1) Create the sightline from the viewpoint to a target point.
(2) Obtain all of the points on the terrain between the viewpoint and the target point.
(3) Calculate the points of intersection between the sightline and the DEM.
(4) Calculate the elevations of these points of intersection using interpolation.
(5) Calculate the elevations of the points on the sightline that correspond with the points of intersection.
(6) Compare the elevations of the sightline with the elevations of the intersecting points to determine the visibility.

The principle of this algorithm is simple but computationally intense. Some algorithms have been developed to reduce the computing effort such as the key slope method (De Floriani *et al*. 1994). This method continually computes the slope along the sightline and updates the max slope, comparing the slope of the current point with the max slope to determine the visibility. After obtaining the slope of the key point on the terrain, it compares all key slopes. All of the curved surfaces on the terrain between two key slopes are considered visible. The required computation was thus greatly reduced with a time complexity of $O(n)$.

Grid DEMs are widely used in many GIS applications and the terrain data in Digital Earth systems often have grids applied, such as NASA's.bil format for height data in NASA World Wind. Many intervisibility algorithms have been developed using gridded DEMs, such as Janus, Dyntacs, ModSAF, and Bresenham. These algorithms are roughly the same except for the interpolation of elevation and the judgment rule for testing visibility.

The Janus algorithm has a high computational efficiency for fewer test points, but it is unable to guarantee that all of the elevation points on the sightline will be considered. It divides the sightline into equal parts and uses four vertices on a grid to perform bilinear interpolation to find the elevation.

The Dyntacs algorithm (dynamic tactical simulation) is similar to the Janus algorithm, but it emphasizes accuracy rather than efficiency. It does not divide the sightline but projects it to the grid to align the points of intersection on the grid edges. It considers the relationship between the slope of the sightline and the slope from the viewpoint to these points of intersection to determine the visibility.

The ModSAF (modular semi-automated forces) algorithm adds diagonals with the same direction for all of the grid cells. It splits each grid cell into two triangles to approximate the surface of the terrain. This algorithm is similar to the Dyntacs algorithm, but not only does it find the points of intersection between the grid edges and the sightline, but it also calculates the intersection of the sightline and diagonals.

The Bresenham algorithm, based on Bresenham's line algorithm for drawing lines in computer graphics, is efficient for its integer arithmetic, but it reduces the calculation precision because of the rounding operation.

2.2 Previous works on viewshed analysis

A viewshed is an area that is visible from a specific location based on the elevation values of a DEM. Essentially, viewshed analysis applies numerous sightlines to find intervisibility between a viewpoint and target points. For efficiency, it does not directly use sightlines to test visibility point by point.

An algorithm by Shapira (1990) traces a line from the viewpoint to any target point along a line of sight. If a grid edge on the DEM intersects with the line of sight before reaching the target point, the target point is invisible; otherwise, it is visible. This algorithm is simple, but it has redundant computations because the sightlines to different target points may overlap partially. This method was improved by Mills *et al*. (1992) by processing the sightlines from the viewpoint in parallel. The elevation data are communicated by two adjacent sightlines in order to reduce the global communications among different processors.

Another algorithm by Blelloch (1990) considers only sightlines from the viewpoint to the target points on the far sides of the destination region. It determines

Downloaded by [Colorado College] at 16:34 14 October 2014

the visibility of all points along a sightline by using the current tangent from the viewpoint to the terrain. This method was improved by Teng *et al*. (1993) by processing all sightlines corresponding to a far side in parallel. The source region is divided into four parts by two lines with a 45-degree slope passing through the center of gravity of the region. Each part is swept from the center to the boundary of the source region. Global communications are reduced by using the coherence between pixel strips processed in consecutive stages of the sweep process.

Another plane-sweep algorithm was proposed by Van Kriveld (1996). It computes extended viewshed through a radial sweep. It applies a sightline rotating around the viewpoint to sweep the whole terrain. During the rotation, the visibility of each point is calculated. Franklin and Ray (1994) proposed an algorithm using an assisting grid. It computes approximated visibility counts on a regular square grid (RSG) by considering a restricted number of sightlines for each viewpoint and a restricted number of target points on each line.

All of the algorithms above have used line-of-sight calculation. An exception is by Wang *et al*. (2000), who presented an algorithm using a reference plane to determine visibility. This reference plane is generated by the spatial relationship between the viewpoint and target point. It is much more efficient than any other algorithm based on lines of sight.

Downloaded by [Colorado College] at 16:34 14 October 2014

2.3 Discussion of traditional algorithms for viewshed analysis

The algorithms above are based on line-of-sight operations, except in the work by Wang *et al.* (2000), and though some algorithms use parallel structure, their efficiency is not high and is restricted by the capability of the CPU on which they are run. The line-of-sight operation is time consuming, using interpolation to compute the elevations of the intersecting points between the sightlines and grids. An increase in the number of grid cells will increase the intersection operation's complexity and reduce the efficiency of the line-of-sight operation. Also, the position of the viewpoint and the range of the viewshed will affect calculation time; expansion of the range will greatly reduce the efficiency. But the calculation time of the algorithm by Wang *et al.* (2000) is not affected by the location of the viewpoint and the range of viewshed, and even increasing the total grid cells has little effect.

The viewshed on an RSG is usually encoded in a discrete way. It marks each grid cell as visible or invisible. The discrete representation will be accurate if the input RSG is dense enough, but if the grid is sparse, a grid cell may be partially visible and partially invisible so that the discrete form causes aliasing problems. Usually, a triangulated irregular network (TIN) is suitable for a continuous visibility map, but considering the terrain's representation, data storage, and some rendering technologies such as level of detail (LOD), gridded DEMs are more preferable for Digital Earth systems such as Google Earth and NASA World Wind, which have terrain data structures based on an RSG.

2.4 Introduction to shadow mapping

Shadow mapping, introduced by Lance Williams in 1978, is a process to generate shadows under a light source in 3D computer graphics. This algorithm is still widely used in real-time scenes in many 3D applications. The principle of 3D rendering is

projecting the whole 3D scene to a 2D plane in the observer's view space. The transformation from 3D to 2D is the essence. By looking at a 3D scene, it is transformed into a '2D picture' in our eyes. Each pixel in the 2D picture represents a visible fragment (not being blocked) in the 3D scene, and this fragment's distance to our eyes is the depth value of this pixel. Looking from the view of the light source (assuming having put a light source in the 3D scene), any visible fragment is in the light. Anything behind these illuminated fragments is in shadow. This is the basic principle of this shadow mapping algorithm, which is composed of two steps. The first is rendering the scene to a texture from the view of the light source. This texture stores the depth information of every fragment it sees. The texture is similar to a gray-scale image, and each point on it stores a gray value that represents the depth information. Here the texture makes up the shadow map. The next step is to render the scene to the screen while comparing the distance from each fragment to the light source with the corresponding depth value encoded in the shadow map. Every fragment is projected to this texture to get its 2D coordinate. The 2D coordinate is used to locate its depth value stored on the texture in the first step. If a fragment's distance to the light source is larger than the corresponding depth value in the shadow map, that fragment is in shadow. Otherwise, it is illuminated. Figure 1 shows a simple example.

In Figure 1, the scene is rendered from the view of the light source and we get a 'shadow map' texture. Then we render the scene to the observer. During this part, each fragment is determined to be in or out of shadow. For example, FF is in shadow because FF's distance to the light source is larger than the PP's depth value stored in the shadow map texture. The shadow map texture was from the first step, so the depth value stored in PP is FN's distance to the light source because FN (not FF) can be seen from the light's point of view. The pseudo-code of the shadow map algorithm is described below:

Step 1: Construct scene from the view of light source: get a shadow map

Step 2: Render scene from the view of observer:

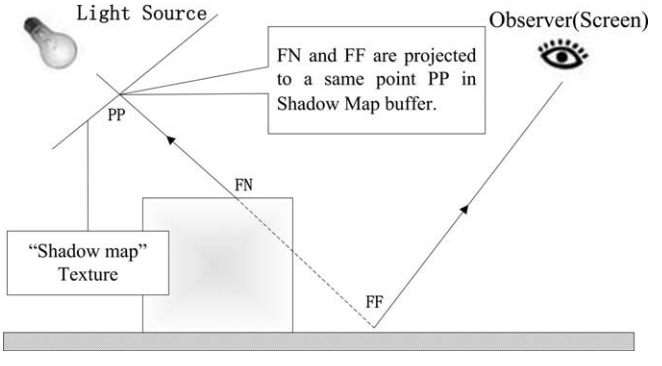


Figure 1. An overview of the 'shadow map' algorithm.

Downloaded by [Colorado College] at 16:34 14 October 2014

For each fragment:
P = fragment's position under the light's space. //project fragment to shadow map//

A = shadow_map(P.xy) //get its depth value stored in shadow map//

B = P.distance_to_light //compute the fragment's distance to the light source//

A < B? Fragment in shadow: Fragment is lit

2.5 Discussions of shadow mapping

Section 2.4 shows the algorithm for generating shadow under a light source. Step 1 is to project the whole scene to a texture with the depth value to generate the shadow map. For different types of light sources, different projection modes can be chosen. For a point light source, the view should be the perspective projection in step 1; for a directional light source, the orthographic projection should be used. No matter what kind of projection is selected, with the limitation of the texture in size and resolution, the accuracy of a shadow map will affect the result.

One of the disadvantages of real-time shadow mapping is that the size and depth of the shadow map determine the quality of the final shadows. This is usually visible as aliasing or shadow continuity glitches. A simple way to overcome this limitation is to increase the shadow map size, but due to memory, computational, or hardware constraints, it is not always possible. Many algorithms have been developed to solve this issue, such as percentage-closer filtering (William *et al*. 1987), perspective shadow maps (Stamminger and Drettakis 2002), parallel-split shadow maps for large-scale virtual environments (Zhang *et al*. 2006) and variance shadow maps (Donnelly and Lauritzen 2006), though they suffer from aliasing problems.

3. The concept of our parallel algorithm

Our parallel algorithm takes a new approach to simulate the process of viewshed analysis by creating a light source at the viewpoint to illuminate and shade the terrain. The traditional concept of viewshed analysis is putting an observer at the location of the viewpoint to scan the terrain to determine the area in or out of view, a concept that is essentially equivalent to our approach because the transmission of light is straight and reversible. With this prerequisite, our algorithm focuses on how to render the terrain with correct lighting and shading information under a specific light source and integrate this rendering module with the viewshed calculation.

In Digital Earth applications like Google Earth and NASA World Wind, 3D scene rendering can be achieved with a programmable 3D visualization pipeline. By taking advantage of GPU programing, Digital Earth applications are able to incorporate more dynamic effects such as rendering seawater, clouds, and shadows. Our algorithm uses vertex and pixel shaders to implement GPU programing. These two shaders are a set of software instructions working on graphics cards to calculate rendering effect with a high degree of flexibility. They are programmable and use complete parallel computing in the GPU. That is the basis of the parallel, real-time algorithm presented in this study.

Downloaded by [Colorado College] at 16:34 14 October 2014

4. The parallel algorithm for viewshed analysis

Section 3 has introduced a new concept to simulate viewshed analysis using the shadow mapping algorithm in Section 2.4 to generate shadows in a 3D scene. Based on this shadow mapping algorithm, our algorithm creates an omnidirectional light source point with a certain illumination range at the location of the viewpoint to illuminate the 3D scene. The areas in and out of the shadow represent the result of the viewshed analysis. The entire process of this algorithm is shown in Figure 2.

There are three steps to accomplish the algorithm.

Step 1

In Section 2.4, a texture target composes the shadow map, but the texture's size limitation causes only the area within a certain range and direction to be projected to the shadow map, meaning this original algorithm is only suitable for a spot light. However, if the viewing azimuth of the viewshed analysis is restricted like the scope of a spot light, the original algorithm can be used to simulate the viewshed analysis. In order to overcome this limitation, our algorithm generates an omnidirectional light source point. We assume a spherical target surface around the light source with a radius of 1.0. The sphere is the virtual shadow map which allows the whole scene to be constructed from the view of the light source. The reason we choose the spherical shape is that the scene is more easily projected from all directions to the sphere's surface. Then the whole 3D scene is directly projected to this sphere target with depth information. When we render the 3D scene, we build up a 3D Cartesian coordinate system with specific origin coordinates, and the position of any fragment is its world coordinate in this Cartesian coordinate system. So this projection process is making the world coordinates of a fragment transform to the space of the light source by subtracting its coordinates with the light's coordinates. It is equivalent to translating the origin coordinates (which means the coordinates of $X=0$, $Y=0$, and $Z=0$) to the point of the light source. Then the coordinates in the light's view are normalized to be stretched to the virtual sphere surface (the length of a normalized vector is 1.0, which is equal to the radius of this virtual sphere). Now the normalized coordinates are spherical coordinates, and the fragment has been projected to the spherical target surface. Each point on the sphere's surface stores its spherical

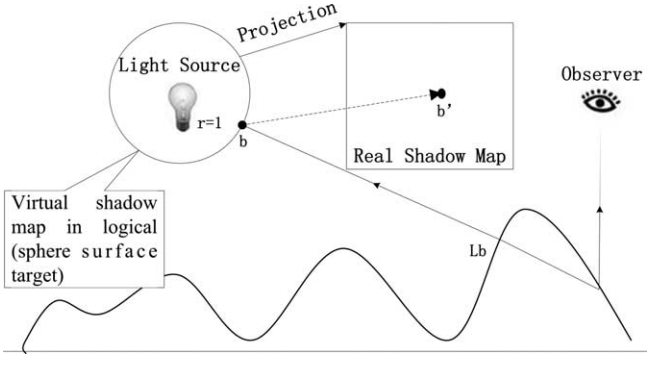


Figure 2. Our improved algorithm.

Downloaded by [Colorado College] at 16:34 14 October 2014

coordinates and depth value of its corresponding fragment in the 3D scene. Here the depth value is the nearest fragment's distance to the light source from a certain projection direction. As shown in Figure 2, point b on the sphere target stores the depth value of fragment Lb's distance to the light source.

Step 2

In step 2, the algorithm transfers the virtual sphere target to a real texture target to generate the shadow map. In a 3D visualization pipeline, the structure of a texture should be a 2D plane with a rectangular shape so that the shadow map generated in step 1 is just a virtual one in logic. In this step, the virtual shadow map is transformed into a real one. We project the sphere's target surface onto a plane texture surface while retaining its depth value. Many projection modes transform 3D spherical coordinates into 2D plane coordinates such as conformal conic projection and equidistant conic projection. In our algorithm, a vertex shader function completes the conformal conic projection (Weisstein 2010). The source code of the vertex shader is described below:

```
float2 StereographicProjection(float3 v3XYZ)

{

float fOneSubZ = 1–v3XYZ.z;

return float2(v3XYZ.x/fOneSubZ, v3XYZ.y/fOneSubZ) * 0.5f;

}
```

It is a simple function to handle vertex transformation. Each call to this function only involves a single input vertex and it is independent of other data. This feature makes it well suited for parallel computing, and the architecture of the GPU is designed to handle it. A vertex shader is designed to process vertex transformation on the GPU, thus this function is executed in parallel with high efficiency.

Step 3

The third step is to render the scene to the screen while using the shadow map generated in step 2 to test the visibility of each fragment. Each fragment is projected to the virtual sphere surface to get its spherical coordinate. This coordinate is transformed to the plane coordinate on the real shadow map texture using the function 'StereographicProjection' with a pixel shader. With the coordinate on the real shadow map, the depth value is obtained. Comparing the depth value with the fragment's distance to the light source, we find its visibility to the light. If the distance value is larger than the depth value stored, the fragment is in shadow and vice versa. Step 3 is processed with the pixel shader, so all of the pixels for rendering are computed to determine visibility. Steps 1 and 2 have generated the shadow map as a continuous texture, and the process to generate this texture also applies a pixel

Downloaded by [Colorado College] at 16:34 14 October 2014

shader. This means that all of the visible points (including the points outside the view range) in the light's view are projected to this texture with their depth information. So when rendering all the pixels to get the whole scene in step 3, every pixel will be tested against the corresponding depth value on the shadow map so that every hidden or visible pixel is rendered with or without shadow. The position and color value of a pixel is generated by interpolation in the GPU. Since all of the pixels are rendered, the result of the viewshed is continuous.

Figure 3 shows all three steps.

The pseudo-code of the vertex shader and pixel shader is shown below.

Passes 1 & 2: generate the shadow map

Vertex shader:

```
{
float4 worldpos = vertex.change_to_light's_viewspace;
out.position.xy = StereographicProjection(normalize(worldpos.xyz));
//function normalize will transform each vertex to the virtual sphere surface.//
out. position.z = worldpos.distance_to_light;
out. position.w = 1;
```

Downloaded by [Colorado College] at 16:34 14 October 2014

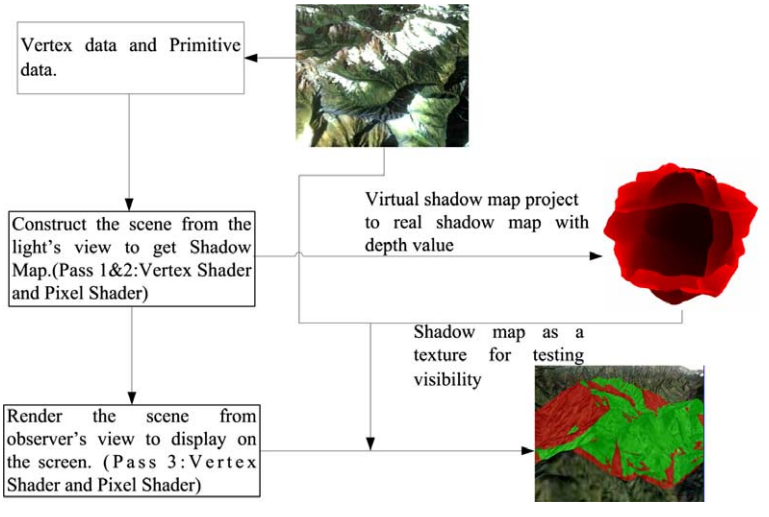


Figure 3. An overview of the process.

```
return out;

}
```

We get the shadow map 'g_texShadowMap' as a texture used in Pass 3. (In the main program, Passes 1 & 2 have set a texture as the render target. Here, it is 'g_texShadowMap.')

Pass 3: render the scene with viewshed analysis

Vertex shader:

```
{

out.pos = vertex.change_to_observer's_viewspace;

//observer's view space is for rendering the 3D scene on the screen//

out.lgtPos = vertex.change_to_light's_viewspace;

return out;

}
```

Pixel shader:

```
{

float2 uv = StereographicProjection(normalize(lgtPos.xyz));

float shadow = lgtpos.distance_to_light > g_texShadowMap(uv).value?0:1;

return diffuseColor*shadow + ambientColor;

}
```

Downloaded by [Colorado College] at 16:34 14 October 2014

5. Advantages of this algorithm

Compared with the traditional algorithms, our algorithm is extremely efficient and works in real time. The most fundamental difference is that it uses GPU parallel programing through the shader language (vertex shader and pixel shader) during the rendering process. The highly parallel structure of GPUs makes them more efficient than CPUs, especially at manipulating and displaying computer graphics. Each register in the GPU stores four float components, and a processor calculates these four elements once at the same time. Some modern GPU architectures consist of hundreds of processor cores that operate together to crunch through the data set in the application (Fernando and Mark 2003). Using many processor cores at the same time, just one instruction on some GPUs can complete work requiring hundreds of

loop functions on a CPU. These remarkable properties of some modern GPUs make the parallel real-time algorithm possible.

In traditional viewshed analysis algorithms, the calculation of visibility for each target point is independent from the rendering module, and the viewshed calculation only involves the input DEM data. But our algorithm has completely blended the viewshed calculation with the rendering module. It uses vertex and pixel shaders through a programmable 3D visualization pipeline to calculate the pixels in and out of shadow and displays the scene with viewshed analysis. That is the real-time advantage of the algorithm.

6. Results and discussion

The algorithm was applied in our 3D Digital Earth system with the DirectX 9.0c API (see Figure 4 for screenshots). It is a real-time application that smoothly regenerates the viewshed with movement of the light source.

This algorithm has been tested on some PCs with different configurations and has captured the results in interactive frame-rates, which are measured with Fraps 3.1.0 (see Table 1). From Table 1, the capability of the CPU does not make much difference, but the graphics card varies markedly. Different graphics cards have GPUs with different capabilities. High-end graphics cards use this algorithm more efficiently because they have more processor cores and faster processing technology. In regard to GPU performance, the graphic card in No. 1 is the best and No. 3 is the worst in Table 1. We can conclude that the most important issue influencing the frame-rates is the capability of the GPU, since the integral part of the algorithm is in vertex and pixel shaders which are not processed using the CPU and system memory (RAM). This means that a high-performance graphic card will accelerate this algorithm.

In the next round of benchmarking, we changed the max visibility scope, which had nearly no effect on efficiency (Figure 5). By contrast, applications using the traditional algorithms usually run more slowly with a larger scope. This is because they emphasize the viewshed computation, not rendering, and the viewshed

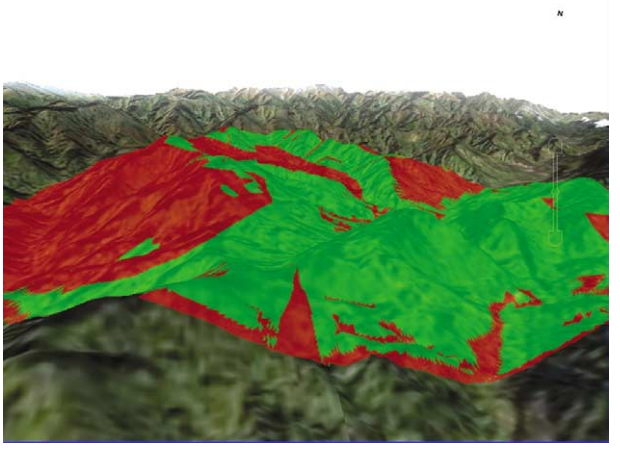


Figure 4. Screenshots of viewshed analysis.

Downloaded by [Colorado College] at 16:34 14 October 2014

Table 1. Testing on PCs with different configurations.

Platform	No. 1	No. 2	No. 3
Overhead	8.4 ms	16 ms	36.3 ms
Detailed configuration	Intel Core2 Q6600 2.4 GHz NVIDIA GTX280 1 GB 3.25 GB RAM	Intel Core2 Quad 2.5 GHz NVIDIA GF9800 1 GB 3.25 GB RAM	Intel Core2 Duo 2.4 GHz NVIDIA GF8600 256 MB 3.25 GB RAM

range directly affects the amount of calculation required (except in the work by Wang *et al.* 2000). But in our algorithm, rendering the scene is the main operation, and viewshed computation has been combined with it, so changing the viewshed range does not affect the generated shadow map. Only in step 3 do more pixels within the increased viewshed range need to be tested to find their visibility. This testing process is just comparing a distance value with a depth value stored in the shadow map and

Downloaded by [Colorado College] at 16:34 14 October 2014

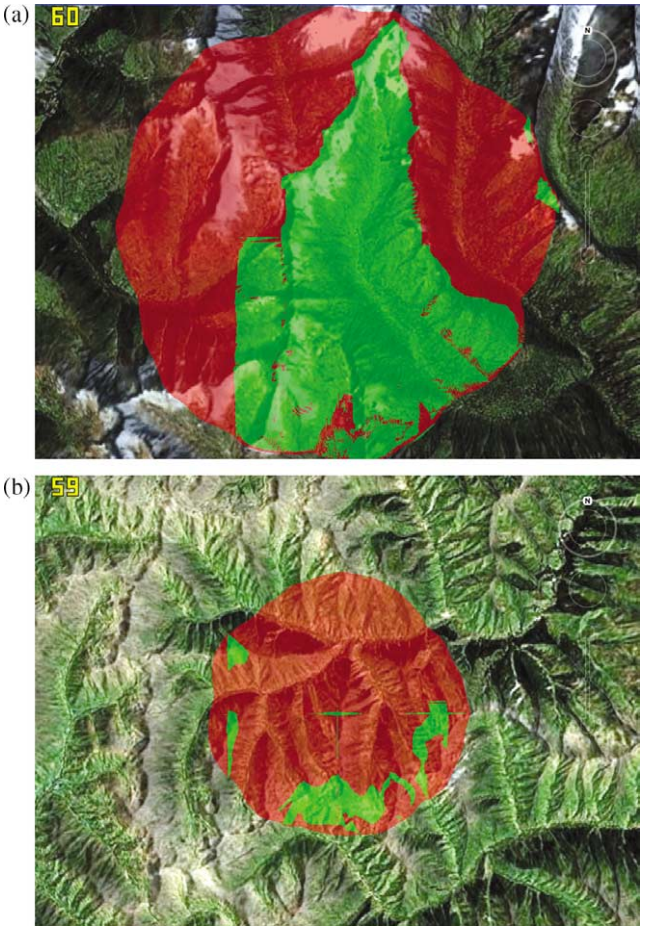


Figure 5. Stable frame-rate with different scopes, (a) scope = 5000 (60 frames/second) and (b) scope = 3000 (59 frames/second).

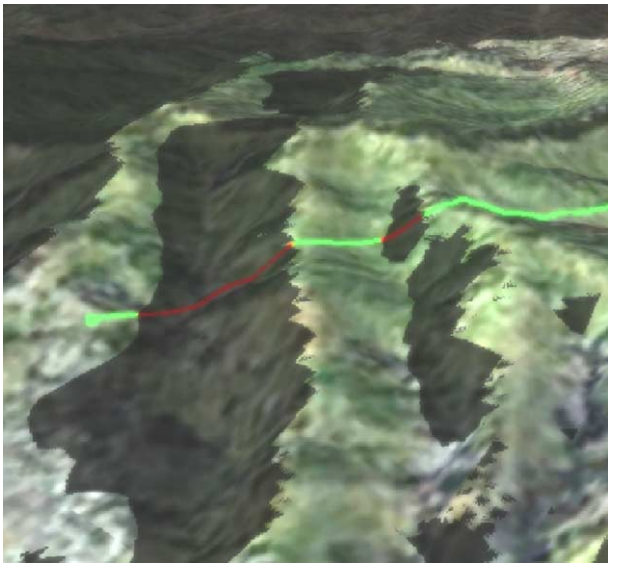


Figure 6. Comparison of intervisibility results. (Available in colour online).

no more computation is needed. That is the reason for its stable efficiency for different visibility scope and real-time performance.

In Digital Earth systems, DEM data may change according to LOD technology. For traditional algorithms, the results of viewshed analyses may also vary with the LOD. Our algorithm also has to face this issue, but has an advantage in that its result must correspond to the rendering of the terrain because this algorithm does not directly handle the DEM data, but the data for terrain rendering. Usually the data for terrain rendering is interpolated from DEM data, and it is usually denser than the DEM data. From a visual sense, the result of our algorithm is more consistent with the terrain.

Compared with the traditional algorithm for intervisibility between viewpoints and target points, our algorithm generates a very precise result (Figure 6). Figure 6 is a local region in the result of the viewshed analysis. The well-lit area is the visible region and the darker region is in shadow. In order to test the correctness of our algorithm, we randomly selected a sightline to apply the traditional intervisibility algorithm that calculates visibility on the sightline. The red section is invisible and the green is visible. Finally, we render this sightline to stick onto the terrain. The result shows that the accuracy of our algorithm can be guaranteed and the method is practical.

Though testing proved the algorithm to be efficient and accurate, some improvements should still be made in the future. A projection from the sphere to the plane is unavoidable in the second step, and the precision of the projection affects the result. In a large area, the farther the region is from the light source (also referring to the viewpoint), the denser the region is in projection (Figure 7). So two separate points may actually become close and even merge into one after projection. A simple solution is to enlarge the projection plane, but this plane is actually a texture which has a size limitation in the 3D pipeline. As high-performance graphics

Downloaded by [Colorado College] at 16:34 14 October 2014

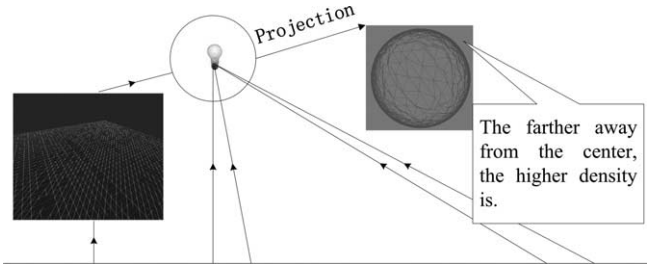


Figure 7. Projection from the sphere to plane.

cards evolve, however, GPUs will be able to support larger textures, and this problem will be improved dramatically even without optimization of the algorithm.

The new algorithm uses a conformal conic projection. The reason is that a conformal projection preserves local angles and shapes, so a straight line remains straight after projection while some other projection methods may bend a line to a curve. In GPUs, a pixel's position and color value is generated by interpolation from some related vertices which constitute triangles around it. So in order to ensure the accuracy of interpolation, the projection must preserve the angles and shapes.

In this algorithm, the projection precision will be affected if the scope and range are too large. Enlarging the shadow map size is a simple solution, but this depends on the graphics card. Most of the graphics cards only support floating precision numbers (only very few high-end graphics cards currently support double precision numbers). We believe that with the evolution of hardware, most graphics cards will support double precision numbers, and the precision of the algorithm will be greatly improved.

7. Conclusion and prospects

The key to this algorithm's efficiency is the two shaders. The shader language was originally used to implement 3D effects during rendering. In this algorithm, the shader language is also used to determine pixel visibility.

Vertex and pixel shaders are highly parallel processes computed on the GPU, and a GPU's parallel computation is not limited to these functions alone. General-purpose computing on the GPU is widely used now for scientific computing, such as NVIDIA CUDA and Microsoft DirectCompute. Taking full advantage of the GPU will allow the CPU to do more computations, especially in distributed computing fields. Many traditional algorithms can be ported to a GPU architecture to take advantage of accelerated, parallel computing.

Digital Earth is a complex and systematic project, and Digital Earth application developers should consider the efficiency of their software on large spatial scales. How to develop highly efficient applications for specific scientific fields is always an important research area. 3D rendering will cost a large overhead in a Digital Earth system, so optimization on rendering is a solution to improve the speed of the entire system. The rendering module is able to involve the capabilities of the GPU to do more complex parallel computing such as the viewshed analysis described in this paper. A small improvement may greatly optimize the Digital Earth system.

Downloaded by [Colorado College] at 16:34 14 October 2014

Acknowledgements

We thank every member in the 3D Group (GeoBeans co. Ltd) for providing useful help. This work is supported in part by 863 program grants 2009AA12Z227, 2009AA12Z215, and also by the MOST Program (Grant No. 2008BAH23B04).

Notes on contributors

Fang Chao obtained his Bachelor degree from Peking University with the major of cartography and geographic information science. Now, he is studying for his PhD degree in the State Key Laboratory of Remote Sensing Science. His research interests include virtual reality, virtual geography environment, 3D visualization in Digital Earth. He focuses on the contribution of physical engine in the virtual geography environment and Digital Earth.

Yang Chongjun is the leader of the State Key Laboratory of Remote Sensing Science, and his research interests include spatial information in WebGIS, Digital Earth, and navigation technology.

Chen Zhuo is the doctor of the State Key Laboratory of Remote Sensing Science, who has interests in the research on 3D real-time rendering and nature user interface.

Yao Xiaojing is studying for her Master degree with the major of WebGIS application and RIA.

Guo Hantao has been working for GeoBeans Company for four years in the department of 3D product. He has accumulated rich experience in 3D rendering and system architecture for Digital Earth system.

Downloaded by [Colorado College] at 16:34 14 October 2014

References

Blelloch, G.E., 1990. *Vector models for data-parallel computing*. 1st ed. Cambridge, MA: MIT Press.

Chang, K.T., 2005. *Introduction to geographic information systems*. 3rd ed. New York: McGraw-Hill.

De Floriani, L., Marzano, P., and Puppo, E., 1994. Line of sight communication on terrain model. *Internaional Journal of Geographical Information Science*, 8 (4), 329–342.

Donnelly, W. and Lauritzen, A., 2006. Variance shadow maps. *In: Proceedings of the 2006 symposium on interactive 3D graphics and games,* March 14–17 2006, Redwood City, California. New York, USA: ACM Press, 161–165.

Fernando, R. and Mark, J.K., 2003. *The Cg tutorial-the definitive guide to programmable real-time graphics*. 1st ed. Boston, MA: Addison-Wesley Professional.

Franklin, W.R. and Ray, C.K., 1994. Higher isn't necessarily better: visibility algorithms and experiments. *Advances in GIS research: sixth international symposium on spatial data handling*, September 5–9 1994, Edinburgh, Scotland. London: Taylor & Francis, 751–770.

Mills, K., Fox, G., and Heimbach, R., 1992. Implementing an intervisibility analysis model on a parallel computing system. *Computer & Geosciences*, 18 (8), 1047–1054.

Shapira, A., 1990. *Visibility and terrain labelling*. Thesis (Master). Rensselaer Polytechnic Institute.

Stamminger, M. and Drettakis, G., 2002. Perspective shadow maps. *In*: *Proceedings of the 29th annual conference on computer graphics and interactive techniques*, July 23–26 2002, San Antonio, Texas. New York, USA: ACM Press, 557–662.

Teng, Y.A., DeMenthon, D., and Davis, L.S., 1993. Region-to-region visibility analysis using data parallel machines. *Concurrency: Practice and Experience*, 5 (5), 379–406.

Van Kriveld, M., 1996. Variations on sweep algorithms: efficient computation of extended viewsheds and class intervals. *In*: *Proceeding 7th international symposium on spatial data*

*handli*ng, August 20–24 1996, Zurich, Switzerland. Utrecht University: Information and Computing Sciences, 15–27.

Wang, J., Gary, J.R., and White, K., 2000. Generating viewsheds without using sightlines. *Photogrammetric Engineering & Remote Sensing*, 66 (1), 87–90.

Weisstein, E., 2010. *Stereographic projection – from Wolfram MathWorld* [online]. Available from: http://www.mathworld.wolfram.com/StereographicProjection.html [Accessed 14 September 2010].

Williams, L., 1978. Casting curved shadows on curved surfaces. *Computer Graphics (Proceedings of SIGGRAPH 1978)*, 12(3), 270–274.

William, T.R., David, H.S., and Robert, L.C., 1987. Rendering antialiased shadows with depth maps. *Computer Graphics (Proceedings of SIGGRAPH 1987)*, 21(4), 283–291.

Zhang, F., *et al*., 2006. Parallel-split shadow maps for large-scale virtual environments. *In: Proceedings of ACM international conference on virtual reality continuum & its applications*, June 14–17 2006, Hong Kong, China. New York, USA: ACM Press, 311–318.

Downloaded by [Colorado College] at 16:34 14 October 2014